

Introduction to Databases - Flipped Classroom FIVE

Query Optimizing, Transaction, Locks and Logs - NEED A LAPTOP FOR THIS!

General Instructions:

- You must have watched all the videos for Week 9 and Week 10, you must have completed the review exercises and completed Quiz 5 BEFORE doing the “flipped classroom” so that you are fully prepared to work on the exercises with your group.
- In your Owl group of students, TOGETHER as a group, discuss and complete the questions below on the paper.
- Take your marked exercises and have one member in your group take a picture with his/her phone of each page. For this flipped classroom, you only need pictures of the front page COMPLETED and pages 7 and 8.
- Have one group member upload and submit the images to owl for the flipped classroom FIVE assignment AFTER IT WAS MARKED. This allows your group to use the flipped classrooms work for studying.
- The goal is for your group to learn from each other, so it is fine, actually encourage, to brainstorm and discuss and problem solve! Feel free to surf the internet or watch the course videos again to figure out your answers.

YOUR GROUP NUMBER: _____

Group Member Name (PRINT)	Present Today (Circle One)	
1	YES	NO
2	YES	NO
3	YES	NO
4 (optional)	YES	NO

This week, in Owl, have one of your group members submit the following:

- The names of the people who attended today in the Owl submission textbox.
- A picture of THIS page and each of your name on the page ((jpg, png or gif format please). Circle who showed up and who didn't
- A group selfie holding this sheet (page 1) of Flipped 5 Instructions up in front with your names written down and circled as to who showed up (jpg, png or gif format please) and looking sad about databases course being over 😞
- Pictures of your results for page 7 and page 8.

ALSO REMEMBER TO HAND IN THIS SHEET TO LAURA!

Objectives:

- To practice making a query run faster
- To see what how transactions work
- To see what happens when a table is locked
- To view log files

QUESTION 1 - Trying out a Transaction

1. Have the person who create the museum database get into mysql OR if that person cannot, then have someone else in your group recreate the database using this code:

<http://www.csd.uwo.ca/~lreid/blendedcs3319/flippedclassroom/four/dbsetup.txt>

2. While in mysql, create two new users: **bart** and **lisa**. Give bart the password **bart** and give lisa the password **lisa**. In mysql, we use the CREATE USER command to create bart and give him the password of bart, so use this command:

```
CREATE USER 'bart'@'localhost' IDENTIFIED BY 'bart';
```

Note: this command means: CREATE USER 'newmysqlusername'@'localhost' IDENTIFIED BY 'newpasswordforthisnewuser';

3. Then use the same command above to create **lisa** with a password of **lisa**.
4. Type the following command to make sure it worked:

```
SELECT User,Host,Insert_priv,password_last_changed FROM mysql.user;
```

notice the date for **bart** and **lisa** is today and notice that we didn't give them any privileges to insert into tables.

Then try this:

```
SELECT User, authentication_string FROM mysql.user;
```

Notice the passwords are encrypted!

5. Press the up arrow key to see those 4 commands you just did again, notice that any commands involving passwords can NOT be repeated via the arrow key, you must type them in again. This is one of the ways mysql hides the password of users, just in case someone walked by your desk when you were at the washroom and pressed the UP arrow on your keyboard. They still will not be able to see the user's password!
6. Do the following command to give bart access to do everything (select, insert, delete, etc) in the museum database BUT only give lisa the ability to SELECT from the workofart table.

```
GRANT ALL ON flipped4db.* TO 'bart'@'localhost';  
GRANT SELECT ON flipped4db.workofart TO 'lisa'@'localhost';
```
7. Now exit from mysql.
8. Open another window to your virtual machine (so that you have TWO windows open to the SAME virtual machine).

9. Log in to mysql in one of the windows as bart and then use the museum database:
`mysql -u bart -p`
`USE flipped4db;`
10. In the second window, log in to mysql as ***lisa*** and then USE the museum database (flipped4db). You should now have 2 windows that are both in mysql, one is bart, one is lisa.
11. In both of the windows, type ***SHOW TABLES;*** Notice that lisa can only see one table.
12. Now we are going to create a transaction where bart adds some rows to the workofart table right at the same moment as when lisa is running a transaction to count how many rows are in the table. Remember that this will cause something we do NOT want, called *Inconsistent Retrievals*. We will put some sleeps in the transaction to slow everything down so that you have enough time to see what is happening.
13. In bart's window type the following:
`SELECT COUNT(woaID) FROM workofart;`
14. In lisa's window also type the above command.
15. Now, in bart's window create the following transaction by copying the code below and pasting it into bart's window. READ IT OVER BEFORE YOU PASTE IT AND NOTICE THAT THE TRANSACTION:
 - a. Shows you the works of art
 - b. Sleeps for 10 seconds
 - c. Inserts 2 new rows into the workofart table
 - d. Shows you the count for the number of works of art
 - e. Sleeps for 1 minute
 - f. Commits the transaction.

```
START TRANSACTION;
SELECT * FROM workofart;
DO SLEEP(10);
INSERT INTO workofart(artname, artist, year, whichmus) VALUES ('Women of Algiers', 'Delacroix', 1834, 1);
INSERT INTO workofart(artname, artist, year, whichmus) VALUES ('Ship of Fools', 'Bosch', 1490, 1);
SELECT COUNT(woaID) FROM workofart;
DO SLEEP(60);
COMMIT;
```

16. While the second DO SLEEP(60); is happening, in LISA's window type the following command and compare the value you got from Barts window to the value you get from Lisas window. Keep doing this command in Lisa's window (hit the up arrow and hit enter) while watching bart's window.
`SELECT COUNT(woaID) FROM workofart;`

17. Notice that Lisa can't see the new value UNTIL the commit happens! Wait till the timer is up, and hit enter for the commit, then go back to Lisa's window and press up arrow and run that select command again. Make sure she now sees the new value.
18. Try copying and pasting the code for Bart, into Barts window, but this time use this code:

```
START TRANSACTION;
SELECT * FROM workofart;
DO SLEEP(10);
INSERT INTO workofart(artname, artist,year,whichmus) VALUES ('Newart1', NULL, 1834, 1);
INSERT INTO workofart(artname, artist,year,whichmus) VALUES ('Newart2', NULL, 1490, 1);
SELECT COUNT(woaID) FROM workofart;
DO SLEEP(60);
ROLLBACK;
```

19. Again, while the select sleep is happening, have Lisa keep running this command in her window (press the up arrow):
`SELECT COUNT(woaID) FROM workofart;`
20. Once the time is up, have Bart run the COUNT query above (the one that Lisa was running) in Bart's window and notice that the 2 insert commands got ROLLED BACK!
21. Exit out of mysql in both windows and close ONE of your windows.

Hopefully this gives you a little bit of insight into Transactions!

QUESTION 2 - Using Log files

1. To keep track of what your users are doing, mysql has log files. In order to use them you need to turn on logging. There are 3 log files
 - a. *general_log* → keeps track of queries
 - b. *error_log* → keeps track of errors
 - c. *slow_query_log* → keeps track of slow running queries
2. Make sure you exit out of mysql so you are at your VM regular prompt. To see if your log files are turned on or not, type the following command (make sure the yellow is the correct password for the root in your mysql):

```
sudo mysql -u root -pcs3319 -se "SHOW VARIABLES" | grep -e log_error -e general_log -e slow_query_log
```

3. This command tells you if the logs are turned on and shows you where the log files are stored.

4. Move to the directory where the general_log files is be kept. Likely the command will be:
cd /var/lib/mysql
5. Do the following command to see all the files in this directory:
ls -la
6. If the log file is there, it will likely be called *cs3319-vm???.log*. If it is there, then do:
sudo more cs3319-vm???.log
to see what it contains (*BUT keep in mind, it might not exist yet so you might need to skip this step*).
7. We will now see the log files in action. Turn on the logging if it was not turned on in step 2 above as follows:
 - a. Get back into mysql as follows:
mysql -u root -p
Then set the logging variable while in mysql as follows:
SET GLOBAL general_log = 'ON';
 - b. Then do USE on one of your databases and do a SELECT on one of the tables
 - c. Exit mysql
8. Still within /var/lib/mysql do
ls -la
Your log file should be there now, likely called *cs3319-vm???.log*
9. Do
sudo more yourlogfile.log
and make sure the SELECT command you just did is there.
10. To start a fresh log, get back into mysql as root and turn off the general_log variable:
SET GLOBAL general_log = 'OFF';
11. Exit mysql and then back up your log file:
sudo mv cs3319-vm???.log cs3319-vm???.log.backup
(make a backup of the old log)
12. Then go back into mysql and turn your general_log variable back on
13. Then do a select or insert to make something happen in the log file
14. While still in mysql, create a new user (any user e.g. 'fred'@'localhost') identified by a password, pick the name of your favourite country as the password for your new user. See step 2 at the very beginning of this flipped class if you cant remember how to make a new user (the way you created users bart and lisa).
15. Then get out of mysql and look at your log file. Noticed that the log file hides the password!
This is another security feature in mysql!
16. The slight problem with log files is that they can generate big files, so, finally, you might want to go now get back into mysql and turn OFF the general_log (just hit the up arrow key

to find the command to turn it on and then make the necessary change to turn it off) since we are just playing around with mysql in this course and don't really need the log files;

You should now have a bit more insight into logging files.

QUESTION 3 - Optimizing Queries

Now we are going to try out some queries to see which ones are fast and which ones are slow and what we can do to speed them up. First, we need to generate a table with lots of data.

1. If you did this step last night to be prepared, you do NOT need to do Step 1, just skip to Step 2 below. If you did NOT do THIS STEP (ALL of Step 1) last night (to get a bit ahead), then you need to do them now: (Please note this could take up to 50 minutes to run!)

a. On one student's virtual machine, do the following commands:

```
cd ~  
mkdir flipped5  
cd flipped5  
nano in.txt
```

b. On your laptop, go to a browser like Chrome and open the following mysql commands and copy ALL of them into your clipboard:

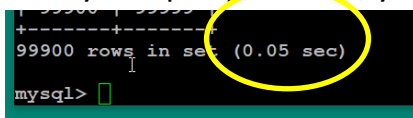
<http://www.csd.uwo.ca/~lreid/blendedcs3319/flippedclassroom/five/setup.txt>

c. Paste the clipboard into your virtual machine screen (in.txt) and then hit CTRL O (and then "Y") to save the file and CTRL X to exit.

d. Type the following command to run the script files (change the yellow part below to your password, likely cs3319). **NOTE: THIS WILL TAKE A LONG TIME TO RUN!**

```
sudo mysql -nvvf --verbose -pyourpassword < in.txt > out.txt 2>&1
```

2. Get into mysql as root
3. Use the flip5 database you just created (USE flip5;)
4. Show the tables in the database flip5. (SHOW TABLES;)
5. Look at the table structure (DESCRIBE product;)
6. Do a SELECT * ... to see that all the data was created in the **product** table.
7. Notice that after your query finishes, you can see the time in seconds that it took to run



```
mysql> SELECT * FROM product;  
+-----+-----+  
99900 rows in set (0.05 sec)  
mysql>
```

your query. Write down the time it took for your first select:

-
8. To return lots of data, we are first going to do a Cartesian product of the big table to itself, i.e.: **product X product**. The problem is that this would return 99,900 * 99,900 rows which KILLED my process (it returns about 10 billion rows). So we are still going to do a Cartesian

product but with less rows. Type in these command (this command will take a 1 minute or more to run!):

```
FLUSH TABLES;
```

```
SELECT * FROM product as a, product as b WHERE a.prodid <1000 AND b.prodid<500;
```

Write down the number of rows you returned (should be close to 1000*500 or 500,000):

_____ and write down the time it took to run: _____

9. Now modify that command slightly. One way to optimize a query is to not return as many **columns**, so change your commands to this and see if the time improves:

```
FLUSH TABLES;
```

```
SELECT a.prodid FROM product as a, product as b WHERE a.prodid <1000 AND  
b.prodid<500;
```

Write down the new time: _____

MORAL OF THIS EXPERIMENT → ONLY RETURN THE COLUMNS YOU REALLY NEED, MORE COLUMNS TAKES MORE TIME!

10. *product* has 4 columns: *prodid*, *oldprodid*, *quantityonhand* and *priceperunit*. Both *prodid* and *oldprodid* are integer columns. Now compare the time for these commands:

```
FLUSH TABLES;
```

```
SELECT MAX(prodid) FROM product;
```

Write down the time: _____

to this commands:

```
FLUSH TABLES;
```

```
SELECT MAX(oldprodid) FROM product;
```

Write down the time: _____

11. Why do you think the first command was faster? Now type this command and see if you can tell the important difference between *prodid* column compared to *oldprodid* column (there are 3 differences, but one is REALLY important):

```
DESCRIBE product;
```

MORAL OF THIS EXPERIMENT WAS → _____

12. Now compare these commands:

```
FLUSH TABLES;
```

```
SELECT oldprodid FROM product WHERE oldprodid <110; Write down the time: _____
```

```
FLUSH TABLES;
```

```
SELECT oldprodid FROM product LIMIT 10; Write down the time: _____
```

MORAL OF THIS EXPERIMENT → Use the LIMIT command if you only need to see a certain number of rows because it stops checking all the other rows as soon as it finds that number of rows.

13. Now compare these commands:

```
FLUSH TABLES;
SELECT a.prodid, b.oldprodid FROM product as a, product as b WHERE a.prodid=b.prodid
AND a.priceperunit < 500; Time: _____
FLUSH TABLES;
SELECT a.prodid, b.oldprodid FROM product as a, product as b WHERE
a.oldprodid=b.oldprodid AND a.priceperunit < 500; Time: _____
```

MORAL OF THIS EXPERIMENT WAS → HOLY CRAP - Indexes REALLY make a difference!

14. Now compare these commands that return the exact same results:

```
FLUSH TABLES;
SELECT a.prodid, a.oldprodid FROM product AS a, product AS b WHERE a.oldprodid =
b.oldprodid AND b.quantityonhand<50; Time: _____
FLUSH TABLES;
SELECT prodid, oldprodid FROM product WHERE oldprodid IN (SELECT oldprodid FROM
product WHERE quantityonhand<50); Time: _____
```

Notice in the second query we limited the rows FIRST before doing the join.

MORAL OF THIS EXPERIMENT WAS → THE WAY YOU WRITE YOUR JOINS CAN GREATLY SPEED UP A QUERY!

15. Now compare these queries:

```
FLUSH TABLES;
SELECT oldprodid FROM product WHERE oldprodid>50000 AND oldprodid<90000;
TIME: _____
FLUSH TABLES;
SELECT oldprodid FROM product WHERE prodid>50000 AND prodid<90000;
TIME: _____
```

Now type in the command:

```
SHOW INDEX FROM product;
```

Look at the column called *Index_type*

MORAL OF THIS EXPERIMENT WAS → LAURA HAS A REASON FOR MAKING YOU LEARN B+TREES...remember when looking for a range of values, you can walk/traverse along the leaves!

16. Now compare these queries:

```
FLUSH TABLES;
SELECT prodid FROM product WHERE oldprodid>=500 OR oldprodid<=5000; Time: _____
FLUSH TABLES;
SELECT prodid FROM product WHERE oldprodid>=500 AND oldprodid<=5000; Time: _____
FLUSH TABLES;
SELECT prodid FROM product WHERE prodid>=500 OR prodid<=5000; Time: _____
FLUSH TABLES;
SELECT prodid FROM product WHERE prodid>=500 AND prodid<=5000; Time: _____
```

MORAL OF THIS EXPERIMENT WAS → AVOID DOING QUERIES WITH OR, but if you have to have an OR try putting an index on the column.

17. Perform these steps and compare these identical queries before and after an index is added:

```
FLUSH TABLES;
```

```
SELECT a.oldprodid FROM product as a, product as b WHERE a.oldprodid = b.oldprodid  
and b.oldprodid < 10000; Time: _____
```

```
ALTER TABLE product ADD INDEX speedy (oldprodid);
```

```
FLUSH TABLES;
```

Then hit up arrow to run the SELECT command for Q17 again Time: _____

MORAL OF THIS EXPERIMENT WAS → If you have a column that you always be searching on...GIVE IT AN INDEX!

18. Finally, perform these steps and compare these identical queries before and after an index is added:

```
FLUSH TABLES;
```

```
SELECT * FROM product ORDER BY quantityonhand; Time: _____
```

```
ALTER TABLE product ADD INDEX faster (quantityonhand);
```

```
FLUSH TABLES;
```

Then hit up arrow to run the SELECT command for Q18 again Time: _____

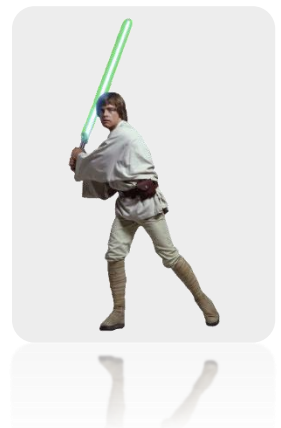
MORAL OF THIS EXPERIMENT WAS → If you have a column that you need to sort frequently, you might want to...GIVE IT AN INDEX!

If you want to learn more, this is the BEST site I have ever seen to teach you about the importance of the database index:

<https://use-the-index-luke.com>

(and I LOVE Star Wars reference 😊!)

NOTE: the FLUSH command was required because mysql is smart and if it finds you querying a table/column over and over again, it starts adding in features to speed things up. I wanted you to see what happens without those feature so that why I kept having you flush out the smart efficiencies that mysql was adding to go back to a clean table.



QUESTION 4 - What Happens When You Lock a Table

1. IF YOU COMPLETED THE STEP WHERE YOU ADD WORKS OF ART for a museum in flipped classroom 4 do the following steps (OTHERWISE jump to step 2 below):
 - a. Then have another group member called G1 (but G1 should NOT be the person who built the museum database) go the php page you built in Flipped Classroom 4 that adds a new piece of art to one of the museums. Thus, G1 should use Chrome to go to the other group members, G2 (the one who built the php pages 2 weeks ago). The site should be something like: <http://cs3319.gaul.csd.uwo.ca/vm???/flipped4/> Click on the **addnewartwork.php** page and add a new work of art.

- b. Next, have the group member G2 (the person who DID build the museum database on their virtual machine) get into mysql again.

```
mysql -u root -p
```

- c. Have that group member G2 do the following commands. This will lock out any users from modifying (writing to) the workofart table:

```
USE flipped4db;
```

```
LOCK TABLES workofart WRITE;
```

- d. At this point, do NOT have the user exit MySQL.
- e. Now have group member G1 try to add another work of art. Write down what happens:

Jump now to step 3.

IF YOU DID NOT COMPLETE THE STEP WHERE YOU ADD WORKS OF ART in flipped classroom 4:

- f. Open 2 windows to your virtual machine

- g. In one window, get into mysql:

```
mysql -u root -p
```

- h. And type:

```
USE flipped4db;
```

```
GRANT SELECT, INSERT ON flipped4db.workofart TO 'lisa'@'localhost';
```

- i. In the second window, get into mysql as follows:

```
mysql -u lisa -plisa
```

- j. In the second window, type the following commands:

```
USE flipped4db;
```

```
INSERT INTO workofart (artname, artist, year, whichmus) VALUES
```

```
("kramer", "seinfeld", 1996, 1);
```

```
SELECT * FROM workofart;
```

- k. In the first window, type the following command:

```
LOCK TABLES workofart WRITE;
```

- l. In the second window, type the following commands:

```
INSERT INTO workofart (artname, artist, year, whichmus) VALUES
```

```
("water", "vanderpas", 2000, 1);
```

Notice it is hanging. Go onto step 3 to fix this.

2. Google to find the mysql command that will unlock the tables.

3. Have the user who is still in mysql unlock the tables and watch what happens to the webpage where you tried to add a new work of art or the second virtual machine.

NOTE: This is how you lock a table in MySQL. Keep in mind that locking tables can cause problems like Deadlock!

Make sure you now go into Owl for Flipped Classroom 5 Assignment and have ONE group member:

- Enter which people in your group showed up today in the Owl text box
- Upload a picture of your group looking sad because today is the last class (well...it is hard to look sad at the end of term but do your best) and holding the front page of this sheet to Owl.
- Upload a picture of pages 7 and 8 with the times filled in.
- HAND IN THIS SHEET TO LAURA BEFORE LEAVING THE CLASSROOM

WHOO HOO! YOU ARE DONE THE FLIPPED CLASSES ☺ . My hope is that you learned more by trying things in our classroom with your classmates than by listening to my boring lectures during class time. HOPEFULLY you liked this way of learning. Let me know here:

<https://sqct.uwo.ca> *(please fill this in, I am supposed to make sure every student evaluates my courses!)*

